```
    volumes:
      - .:/usr/src/app
    environment:
      DATABASE_HOST: database
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: some-long-secure-password
      POSTGRES_DB: myapp_development

  redis:
    image: redis

  database:
    image: postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: some-long-secure-password
      POSTGRES_DB: myapp_development
```

In the pre-Docker world, we'd normally set DATABASE_HOST to localhost since the database would be running on our local machine. Here, though, we specify the name of our service that runs Postgres: database. This resolves to our database service's containers thanks to DNS provided by our app's network.

We also set the POSTGRES_USER, POSTGRES_PASSWORD, and POSTGRES_DB environment variables to match those set for the database service; this means our web service will have the correct credentials to log into the database.

This should now work, but notice that we now have quite a lot of environment variables, and two are duplicated across the web and database services. We also said we'd rather not include secrets in our docker-compose.yml file so we can commit it to our source repo. Let's kill two birds with one stone and extract the environment variables into separate files.

First, let's create some directories to store our environment-specific config:

```
$ mkdir -p .env/development
```

Then create the file .env/development/web (without a file extension), which contains our web-service-specific environment variables:

```
DATABASE_HOST=database
```

and another file, .env/development/database, containing those for our database service:

```
POSTGRES_USER=postgres
POSTGRES_PASSWORD=some-long-secure-password
POSTGRES_DB=myapp_development
```